EX14 - Diccionarios y sets - Configuracion con nombres

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex14_simulation_config_dict_set.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Sustituir muchas variables de configuracion y resultados por diccionarios con claves descriptivas y validar las claves obligatorias con sets.

Variables / elementos del problema

Nombre	Significado
config	diccionario de entradas
results	diccionario de resultados
required/provided	sets de claves

Ecuacion o logica en papel

```
missing = required - provided
```

Comprobacion manual

Si required={"h0","v0","g"} y provided={"h0","g"}, missing={"v0"}.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- funciones
- listas
- tuplas
- validacion

Nuevo en este ejercicio:

- dict
- clave:valor
- config["g"]
- keys()
- set
- diferencia de conjuntos
- resultados etiquetados

3. Conceptos Python nuevos

1. Diccionario

Sintaxis: `config = {"g": 9.81, "dt": 0.1}`

Un dict asocia claves con valores. La clave describe que significa cada dato.

2. Acceso por clave

Sintaxis: `config["g"]`

Los corchetes recuperan el valor asociado a la clave.

3. Set

Sintaxis: `required = {"h0", "v0", "g", "dt"}`

Un set almacena elementos unicos y es util para pertenencia y operaciones de conjuntos.

4. Diferencia

Sintaxis: `missing = required - provided`

Devuelve lo que esta en required pero no en provided.

4. Codigo paso a paso

Paso 1. Crear config

```
config = {
    "h0": 0.0,
    "v0": 50.0,
    "t0": 0.0,
    "g": 9.81,
    "dt": 0.1
}
```

Las entradas quedan agrupadas y etiquetadas en un objeto.

Paso 2. Definir claves obligatorias

```
required = {"h0", "v0", "t0", "g", "dt"}
provided = set(config.keys())
```

config.keys() obtiene las claves del dict; set(...) las convierte en conjunto.

Paso 3. Buscar faltantes

```
missing = required - provided
if missing != set():
    return False
```

Si falta alguna clave, missing no estara vacio.

Paso 4. Usar valores por clave

```
h.append(config["h0"])
euler_step(..., config["g"], config["dt"])
```

Ya no dependemos de recordar posiciones: el nombre de la clave dice que dato usamos.

Paso 5. Devolver resultados etiquetados

```
results = {
    "max_altitude": max_h,
    "final_velocity": v_f,
    "time_of_flight": t_f,
    "iterations": i
}
```

El caller puede acceder a results["max_altitude"] en lugar de recordar un indice.

5. Flujo de ejecucion

```
config dict
 -> required/provided sets
 -> missing check
 -> simulation using config
 -> results dict
```

6. Resultado / comprobacion esperada

La fisica se mantiene igual que EX13, pero entradas y resultados principales quedan identificados mediante claves.

7. Errores tipicos al empezar

- Confundir {} de dict con {} de set.
- Escribir config[g] sin tener g como variable clave.
- Pensar que un set sirve para conservar orden.
- Restar los sets en el orden equivocado.

8. Si solo recuerdas cinco cosas

- dict relaciona claves y valores.
- config["key"] recupera un valor.
- keys() da las claves.
- set compara grupos unicos.
- required - provided encuentra lo que falta.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.